

Kernax

Programmer's Guide

Ziad Iziz

INFOB318 — UNamur

2026

Table des matières

Chapitre 1

Contexte et objectifs techniques

1.1 Pourquoi JAX ?

Le choix de JAX plutôt que pytorch a été motivé par l'optimisation des paramètres des kernels. JAX permet d'appliquer `valueandgrad` direct sur le log vraisemblance, ce qui nous simplifie l'implem.

1.2 Limites connues

Mon SVM n'est pas instancié comme `pytree`, erreur, ce qui veut dire qu'on limite les possibilités de jit.

Chapitre 2

Environnement de développement

2.1 Stack

Nous utilisons principalement Python 3.10+, jax et optax. Pour la doc nous utilisons MystNB avec sphinx.

2.2 Générer la documentation

La documentation est généré avec sphinx et hébergé avec RTD. Pour la générer localement, on place les fichiers dans source puis on "make html".

Chapitre 3

Architecture globale

3.1 Vue d'ensemble

Le projet va tourner autour des KRR, GP et SVM. Un kernel va être un objet qui va pouvoir être appelé et qui va return une matrice de GRAM.

3.2 Le pattern pytree

JAX va exiger que les self soit des pytree pour être jit compiled

Chapitre 4

Description des modules

4.1 GP.py

Implémentation classique d'un GP

4.2 SVM.py

Implementation simple d'un DUAL et primal problème

4.3 KRR.py

Implémentation du KRR, qui est assez similaire au GP mais sans bruit